

Project Case study - Renting

Julien DERAMAIX - Zakaria KADDOUR-BETCHIM - Leo RASSINDRAME - Valentin RICARDO - Aaron RANDRIANARISONA

Introduction

La discrimination dans les processus de location, qu'il s'agisse de logements, est un sujet d'actualité qui mérite une attention particulière. Notre travail consiste à identifier les niveaux à travers l'analyse d'un jeu de données représentant ces processus.

L'objectif est d'explorer plusieurs questions clés :

- Quels sont les chemins suivis par les demandes de location, et leur fréquence ?
- Quels facteurs pourraient conduire à des discriminations, en termes de temps, coût ou accès ?
- Comment ces discriminations se manifestent-elles dans les différentes étapes du processus ?

À travers des analyses sur ce jeu de données, nous visons à sensibiliser aux différents biais et à proposer des recommandations de locations favorisant la réussite de l'échange.

Dataset Renting

Le jeu de données utilisé pour cette analyse porte sur le processus de location de logements. Il a été collecté dans le but d'étudier les variations dans le traitement des demandes et de détecter d'éventuels comportements discriminatoires ou anomalies au sein de ce processus.

Origine

Les données utilisées proviennent d'une simulation conçue pour représenter des scénarios plausibles dans le secteur de la location immobilière. Trois journaux d'événements distincts ont été créés pour modéliser des processus typiques de location, intégrant des facteurs potentiellement générateurs de biais.

Le processus simulé débute par la visite du bien, avec la signature du contrat, l'aménagement dans le logement, jusqu'à la fin de la location.

Degré de discrimination

Les trois journaux d'événements ont été conçus pour refléter différents niveaux de discrimination de la part des propriétaires :

Niveau	Description
Faible	Les décisions sont ouvertes, et des personnes aux profils variés sont acceptées sans restriction notable.
Modéré	Les décisions sont modérées, avec des critères plus sélectifs mais pas systématiquement exclusifs.

Niveau	Description
Élevé	Les décisions sont strictes, avec des critères rigides qui excluent systématiquement certains profils.

Structure des données

Chaque journal (log) est constitué d'un ensemble de traces, chacune contenant plusieurs événements. Les fichiers sont aux format **.XES**.

Voici la structure des traces et des événements des trois ensembles de données :

Il est important de noter que la propriété **religious affiliation**, qui aurait pu être intéressante à inclure, ne fait pas partie des ensembles de données. Cette exclusion a probablement été faite pour éviter toute controverse et peut causer des erreurs dans la conclusion de notre analyse.

"The logs contain attributes that can shed light on potential biases in the process. 'Age', 'citizen', 'German speaking', 'gender', 'religious affiliation', and 'yearsOfEducation' might influence the rental process, leading to potential discrimination."

Source : <https://www.pads.rwth-aachen.de/cms/PADS/Forschung/Event-Logs/~bcbswu/-Un-Fair-Event-Logs>

```
<?xml version="1.0" encoding="utf-8" ?>
<log xes:version="1849-2016" xes:features="nested-attributes" xmlns="http://www.xes-standard.org/">
  <extension name="Concept" prefix="concept" uri="http://www.xes-standard.org/concept.xesext" />
  <extension name="Time" prefix="time" uri="http://www.xes-standard.org/time.xesext" />
  <string key="origin" value="csv" />
  <trace>
    <string key="concept:name" value="1" />
    <boolean key="german speaking" value="false" />
    <int key="yearsOfEducation" value="8" />
    <int key="age" value="25" />
    <boolean key="gender" value="true" />
    <boolean key="citizen" value="false" />
    <boolean key="protected" value="false" />
    <boolean key="married" value="false" />
    <int key="@@case_index" value="0" />
    <int key="case" value="1" />
    <event>
      <string key="activity" value="Apply for Viewing Appointment" />
      <string key="resource" value="Hotline" />
      <date key="time" value="2015-01-05T00:00:42+00:00" />
      <string key="concept:name" value="Apply for Viewing Appointment" />
      <date key="time:timestamp" value="2015-01-05T00:00:42+00:00" />
      <int key="@@index" value="0" />
    </event>
    <event>
      <string key="activity" value="Set Appointment" />
      <string key="resource" value="Real Estate Agent 5" />
      <date key="time" value="2015-01-05T22:13:57+00:00" />
      <string key="concept:name" value="Set Appointment" />
      <date key="time:timestamp" value="2015-01-05T22:13:57+00:00" />
      <int key="@@index" value="1" />
    </event>
  </trace>
```

Entité : Trace

Nom de la propriété	Description
@@case_index	Identifiant unique pour chaque trace

Nom de la propriété	Description
case	Nom ou identifiant du cas spécifique
concept name	Description ou nom de l'activité ou du concept étudié dans le cas
yearsOfEducation	Nombre d'années d'éducation
german speaking	Indique si la personne parle allemand
age	Âge de la personne
gender	Genre de la personne considérée comme homme ou femme (valeur booléenne)
citizen	Statut de citoyenneté
protected	Indique si la personne bénéficie d'une protection juridique particulière.
married	Indique si la personne est mariée

Entité : Event

Nom de la propriété	Description
@@Index	Identifiant unique lié à une trace.
activity / concept:name	Nom de l'activité ou du concept
resource	La ressource utilisée
time / time:timestamp	L'heure à laquelle l'activité est effectuée

Après une analyse visuelle des différentes versions du dataset, nous avons constaté que certaines données présentent des attributs supplémentaires, manquants ou des doublons, comme c'est le cas avec les clés `time` et `time:timestamp`. Bien qu'il n'y ait aucun impact sur la logique, cela peut légèrement affecter la compréhension et le traitement des calculs.

Les statistiques du dataset

Dans un premier temps, dans notre étude de cas, nous visons à obtenir une meilleure compréhension du jeu de données mais également souhaitons guider notre analyse vers des premières pistes potentielles de discrimination dans le dataset. Pour cela, effectuer une analyse statistique serait de première utilité.

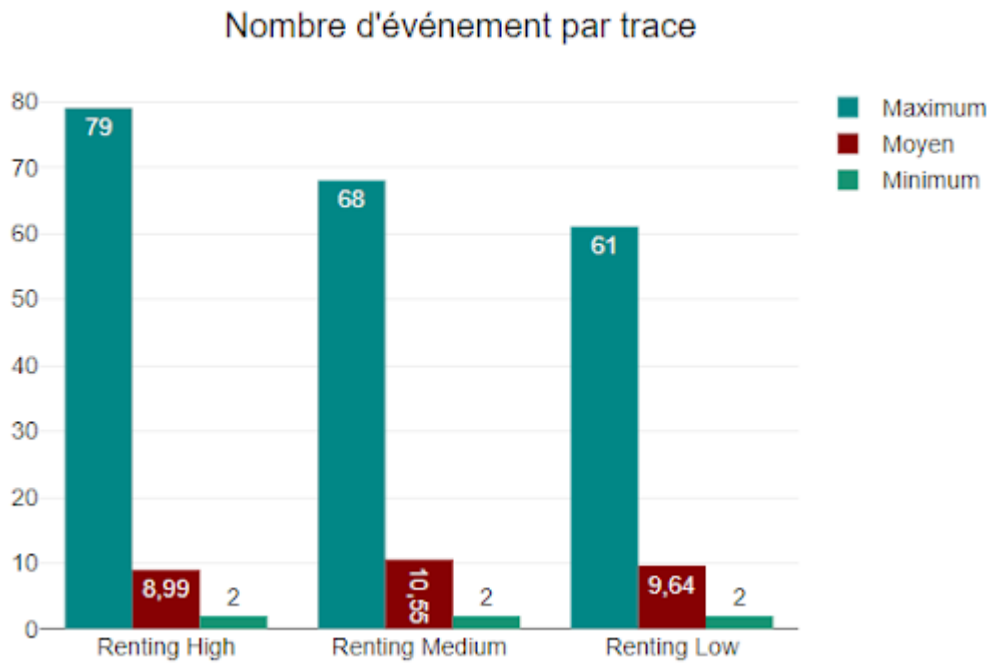
Vue d'ensemble

Il y a avant tout, des données communes à nos trois datasets :

Nombre total de traces: 10 000

Nombre de types d'activité unique: 16

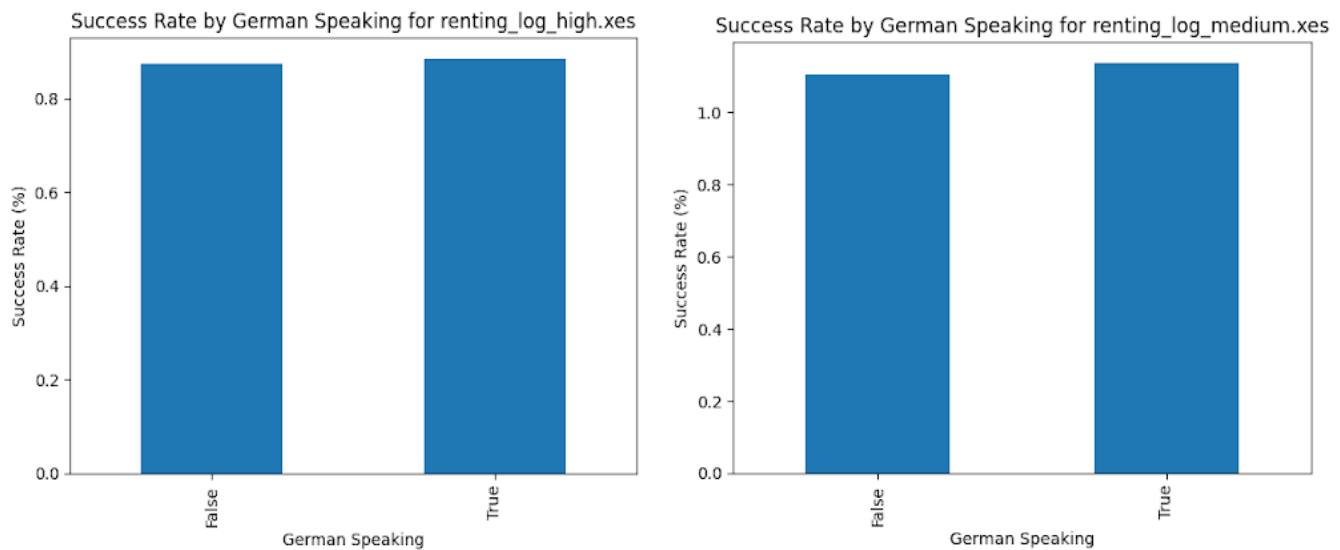
Nombre total d'activités effectuées dans l'ensemble des datasets: 292 000

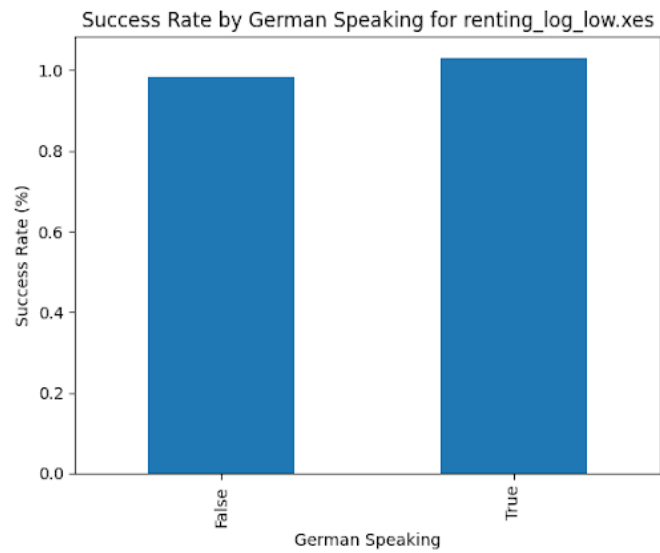


Taux de succès par propriétés du dataset

Nous allons observer pour chaque propriété du dataset le taux de personnes qui signent une location. Ces données nous permettront d'extraire de potentielles propriétés discriminantes. Cependant, la discrimination pouvant être multi-factorielle, les conclusions faites sur ces statistiques ne seront pas représentatives des vraies sources de discrimination et ce sera donc en addition avec le clustering que l'on obtiendra de meilleures conclusions.

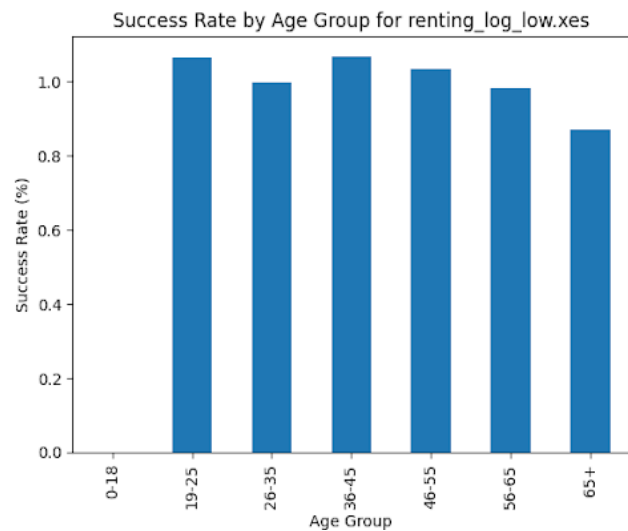
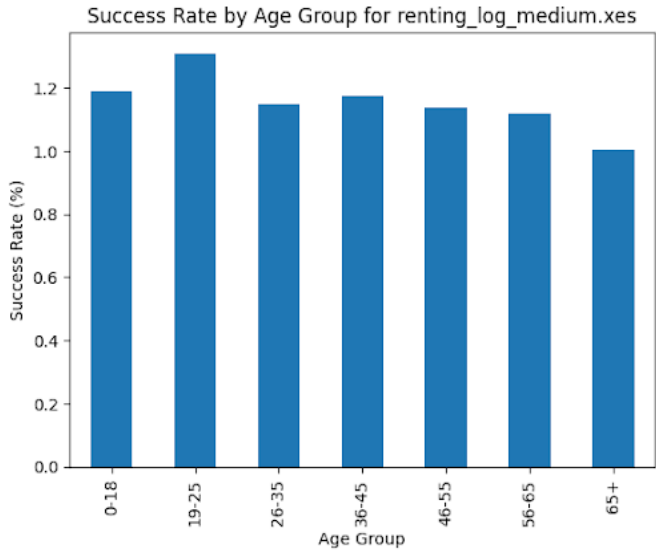
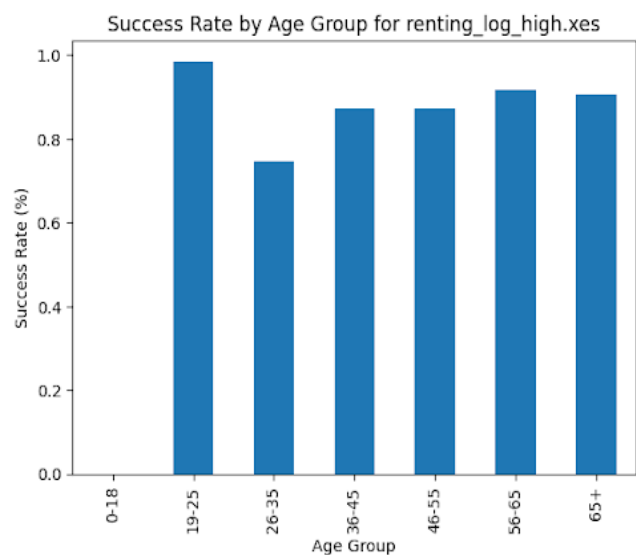
Germanophone





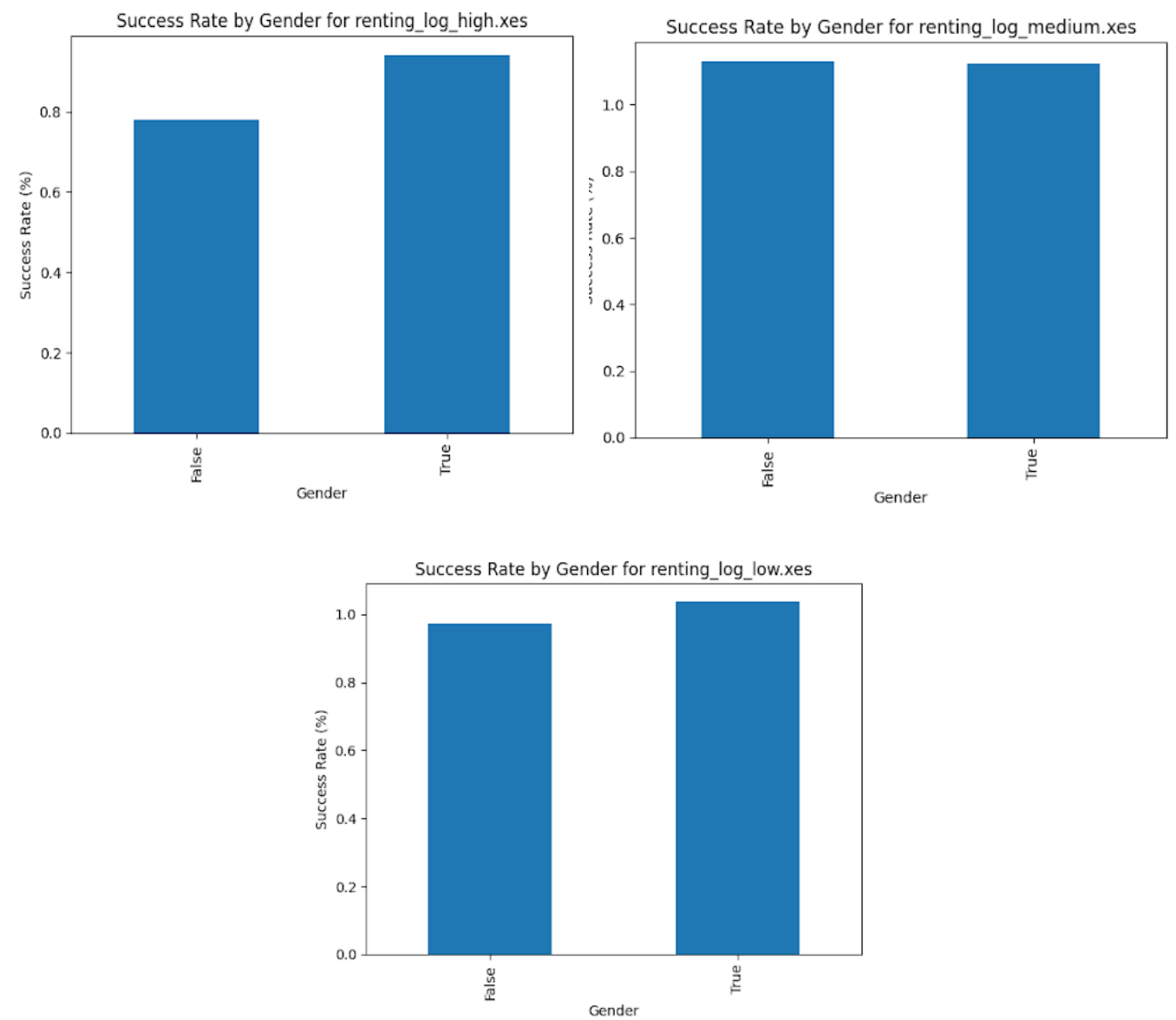
Sur ces graphiques, on observe une tendance où les non-germanophones sont légèrement plus refusés. Ainsi on qualifie cette propriété comme légèrement discriminante.

Groupe d'âge



Ici, on observe de manière globale une baisse des signatures pour les personnes de plus en plus âgées. Cette propriété est donc discriminante.

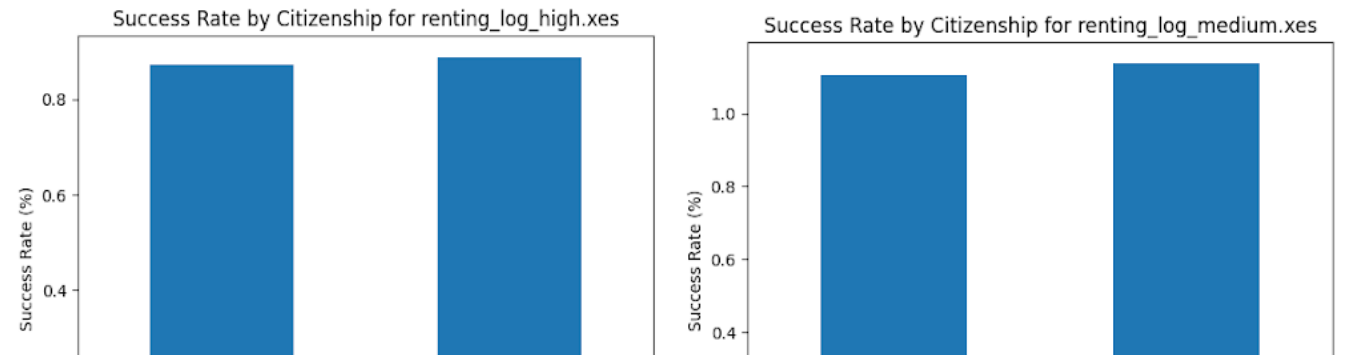
Genre (H/F)

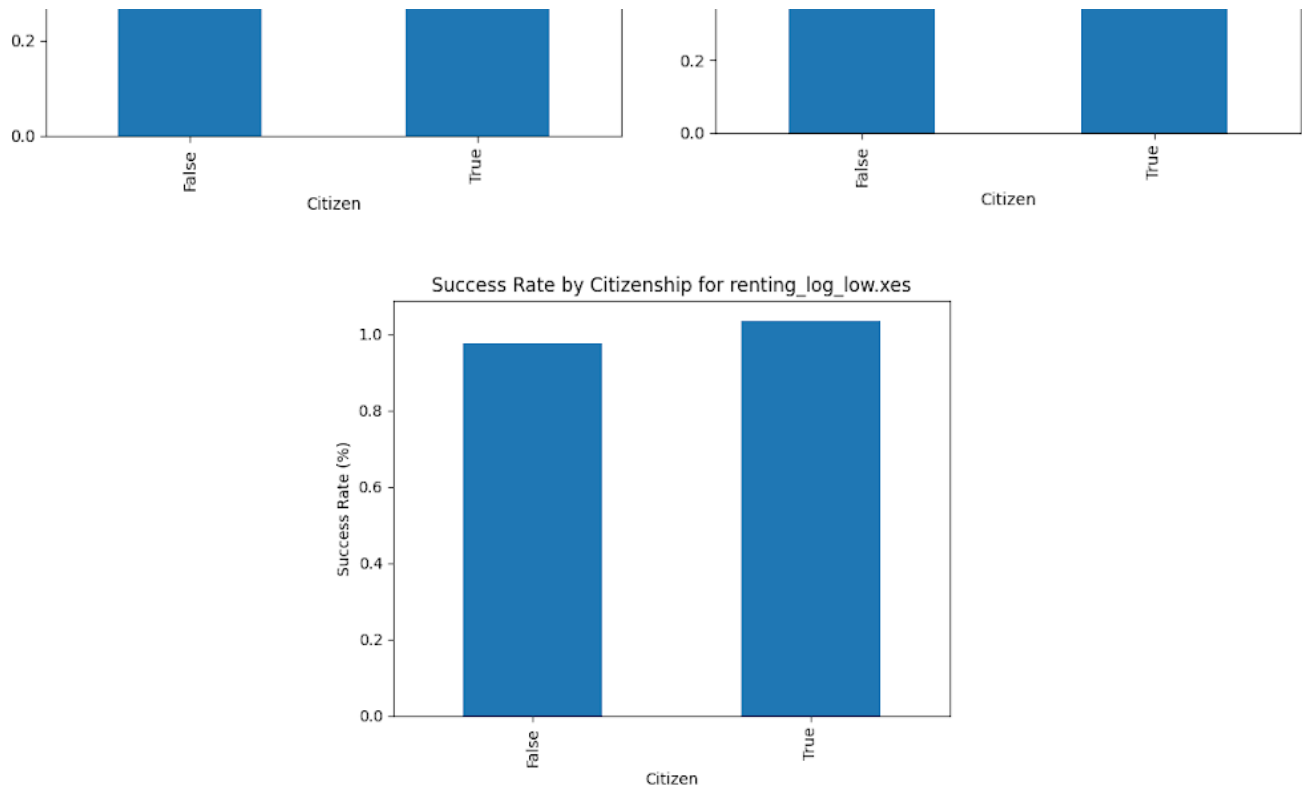


Nous avons décidé qu’un homme est symbolisé par True et une femme par False.

Ainsi, ici on observe moins de signatures pour les femmes, on classe donc cette propriété comme discriminante.

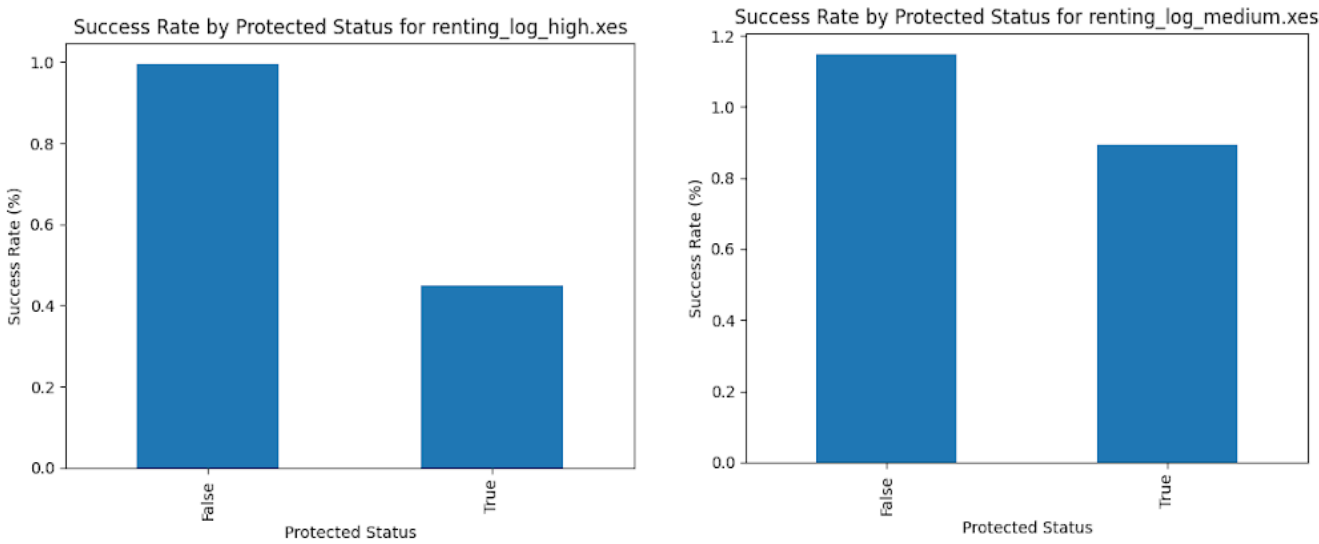
Citoyenneté





Encore une fois, on observe une différence entre ceux qui ont la nationalité et ceux qui ne l'ont pas. Cette propriété est donc légèrement discriminante.

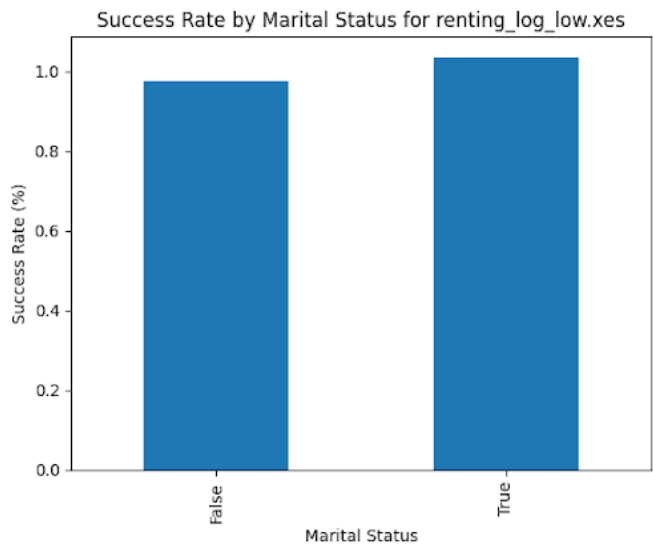
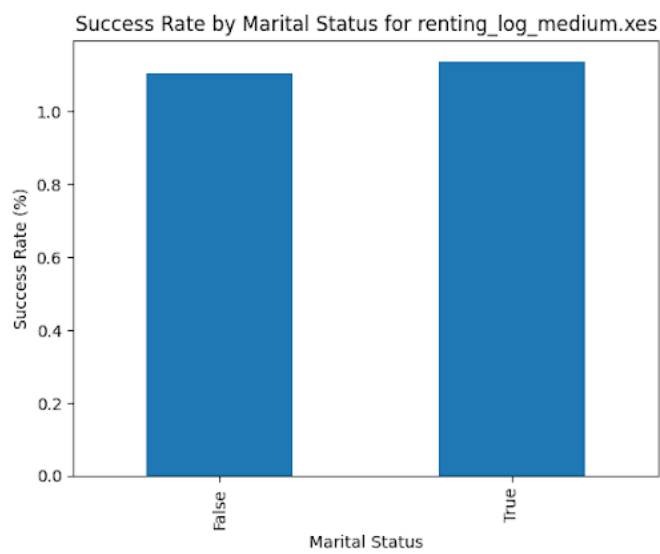
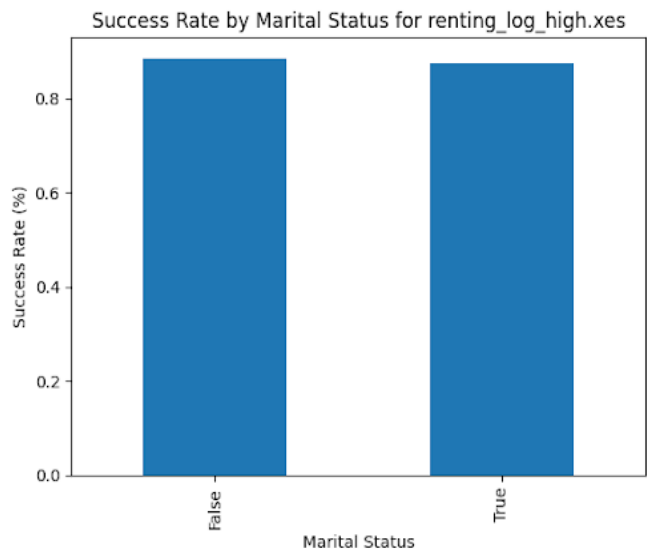
Statut protégé





Ici les observations sont très claires, les possesseurs du statut "protected" signent moins souvent que ceux qui ne l'ont pas.
La propriété est clairement discriminante.

Statut marital



On observe qu'être marié augmente légèrement les chances de signer le contrat de location, donc la propriété est légèrement discriminante.

Conclusion statistiques

Après cette vue d'ensemble, on a noté sur ce tableau les propriétés légèrement discriminantes en bleu, tandis que celles nettement discriminantes en rouge.

Nom de la propriété	Description
yearsOfEducation	Nombre d'années d'éducation
german speaking	Indique si la personne parle allemand
age	Âge de la personne
gender	Genre de la personne considérée comme homme ou femme (valeur booléenne)
citizen	Statut de citoyenneté
protected	Indique si la personne bénéficie d'une protection juridique particulière.
married	Indique si la personne est mariée

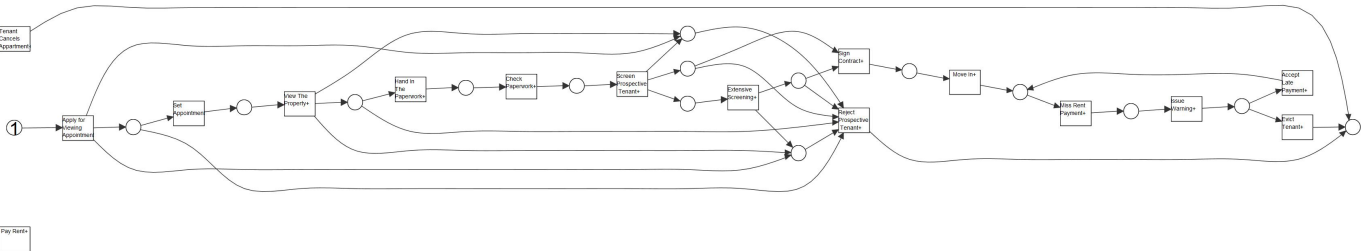
Algorithme de découverte

Après une rapide vue d'ensemble, nous avons appliqué différents algorithmes (Alpha, heuristiques, induction et fuzzing) connus sur nos jeux de données. Ces algorithmes nous permettent d'analyser en profondeur les processus et d'en extraire des modèles.

Alpha



Capture d'écran du réseau de Petri généré par PM4PY sur le dataset *renting_Log_medium*



Capture d'écran du réseau de Petri généré par ProM sur le dataset *renting_Log_medium*

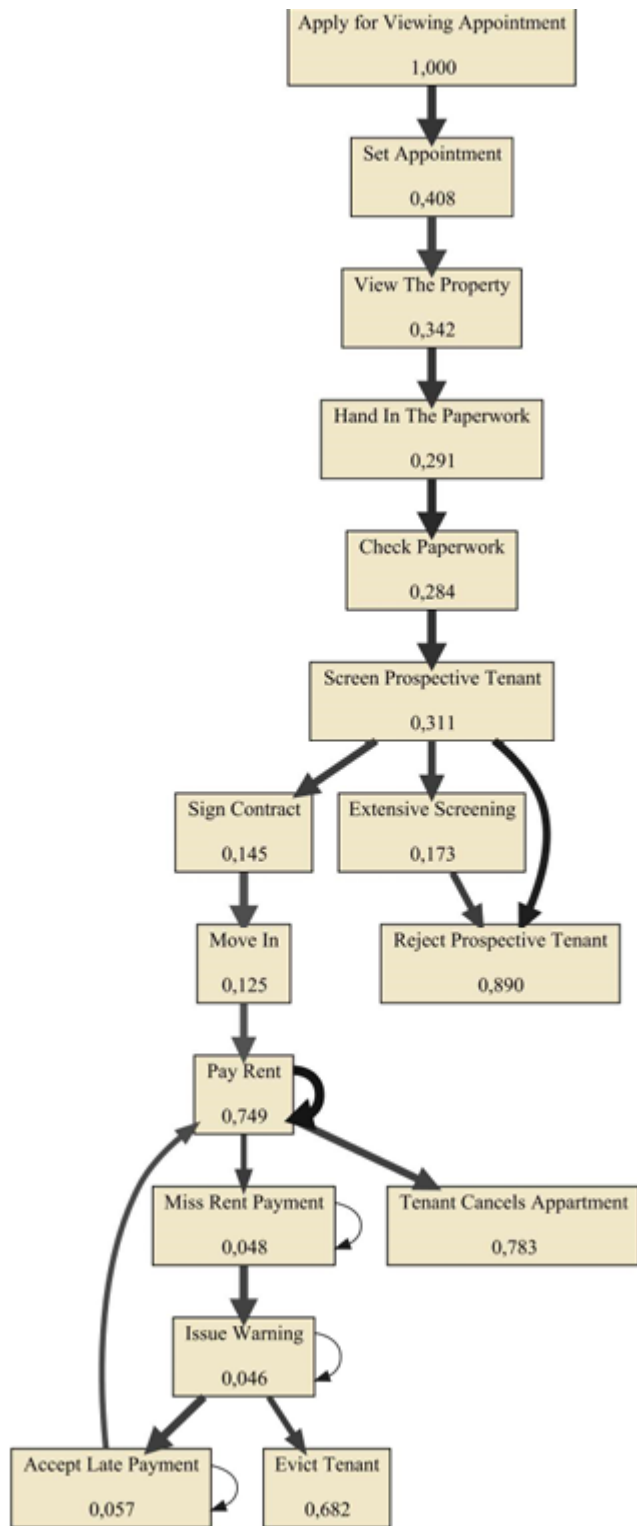
Métrique	High	Medium	Low	Moyenne
Fitness (TBR)	0.77	0.76	0.76	0.76
Precision (TBR)	0.36	0.36	0.36	0.36
Generalization	0.98	0.98	0.98	0.98
Simplicity	0.62	0.62	0.62	0.62
Precision	0.55	0.55	0.54	0.54
Recall	0.97	0.98	0.97	0.97
F-score	0.70	0.70	0.70	0.70

- **Fitness (0.76)** : Son fitness est le plus faible des trois, indiquant une capacité limitée à reproduire les données observées. Cela peut poser problème dans des cas nécessitant une haute adéquation.
- **Precision (0.54)** : Sa faible précision montre qu'il est susceptible de produire des comportements non observés dans les données, ce qui peut nuire à la qualité du modèle.
- **Generalization (0.98)** : Il excelle dans cette métrique, indiquant qu'il génère des modèles très généralisables, capables de capturer les comportements futurs potentiels tout en évitant le surapprentissage.
- **Simplicity (0.62)** : Il est également le moins performant en simplicité, ce qui pourrait rendre ses modèles plus complexes et moins intuitifs.
- **Recall (0.97)** : Avec un rappel très élevé, il couvre la majorité des traces observées, garantissant que les modèles qu'il produit ne laissent pas de comportements non représentés.
- **F-Score (0.7)** : Son équilibre entre précision et rappel est limité par ses performances relativement faibles sur certaines métriques.

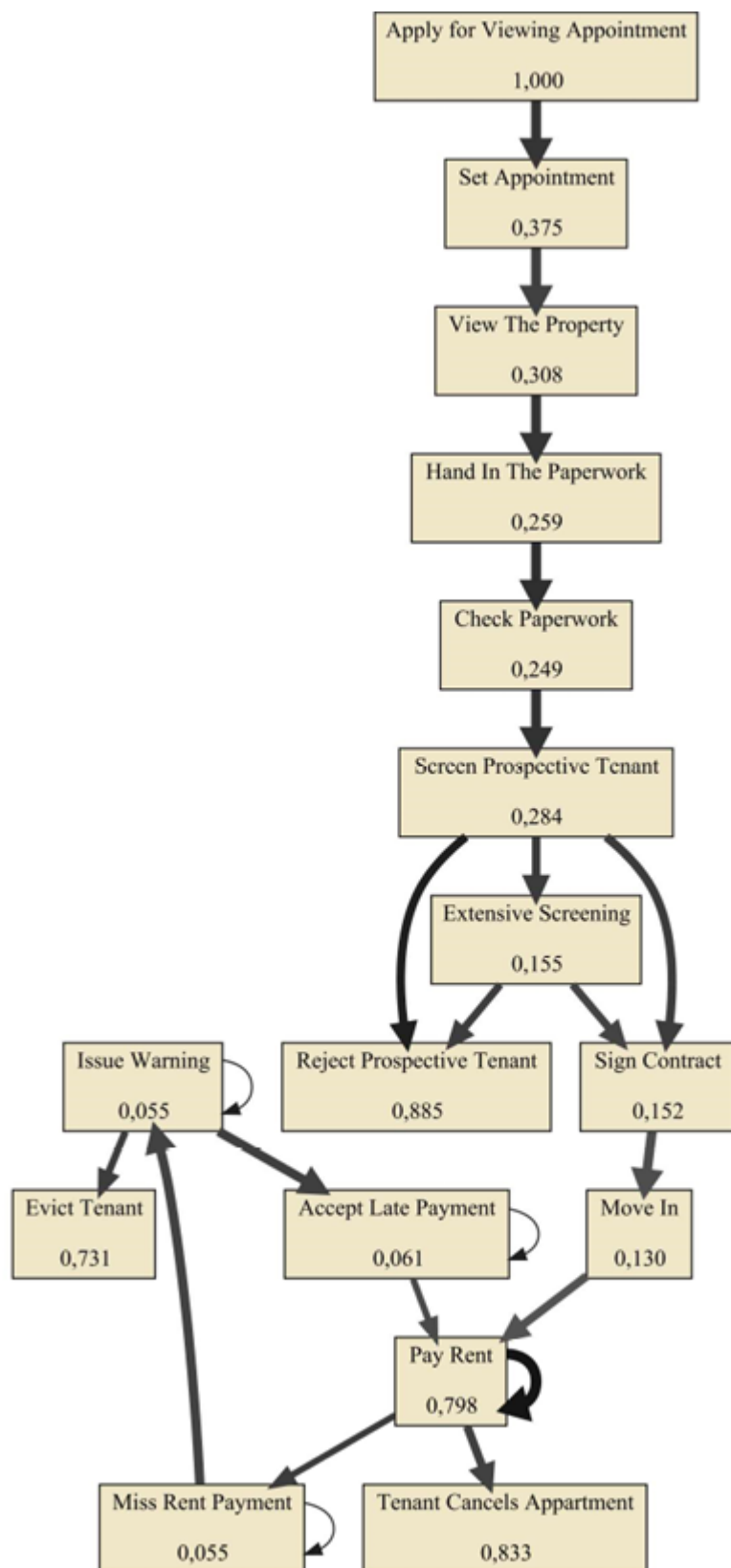
L'Alpha Miner (α -Algorithm) est un algorithme simple et intuitif, efficace pour modéliser des relations simples, comme lorsqu'une personne visite un bien et se désiste peu après. Cependant, il peine à représenter des comportements plus complexes, tels que des événements répétitifs comme "Pay Rent", qui se renouvelle régulièrement dans les logs. En raison de cette limitation, l'Alpha Miner est particulièrement adapté pour des applications nécessitant une forte généralisation et un bon rappel, mais il montre ses limites dans les scénarios demandant une haute précision ou une représentation des répétitions d'événements. Pour mieux capturer ces comportements complexes, il est recommandé d'utiliser un algorithme plus avancé, tel que l'Inductive Miner.

Fuzzer

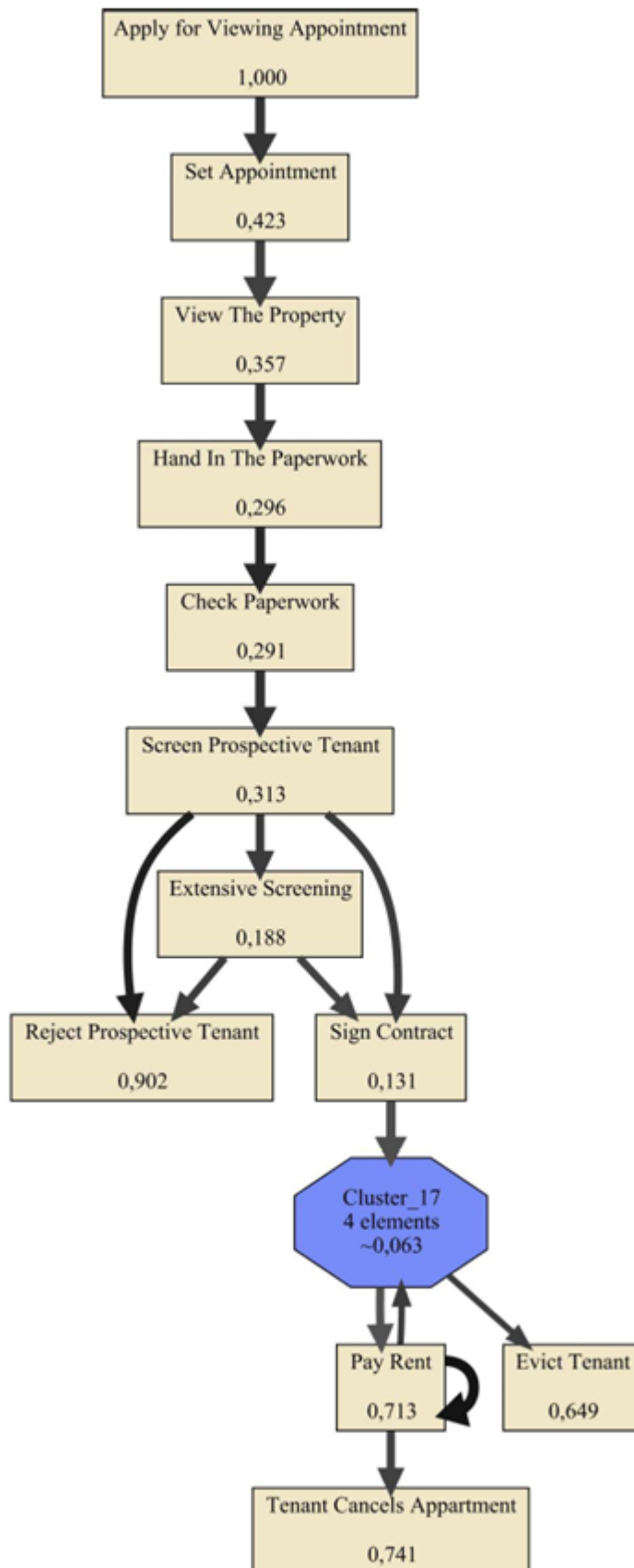
Le Fuzzer Miner est un algorithme de mining de processus qui utilise une approche exploratoire basée sur le "fuzzing". Il perturbe aléatoirement les événements et relations dans les logs pour découvrir de nouveaux modèles de processus, y compris des comportements complexes ou inattendus. Bien qu'il soit robuste face aux anomalies dans les données, il peut produire des modèles plus difficiles à interpréter et moins précis que des algorithmes classiques. Il est utile pour tester la résilience des processus et identifier des comportements non observés dans les logs.



Capture d'écran du fuzzy model généré par Fuzzer Miner avec ProM sur le dataset *renting_Log_Low*



Capture d'écran du fuzzy model généré par Fuzzer Miner avec ProM sur le dataset *renting_log_medium*



Cluster 17:



Heuristics

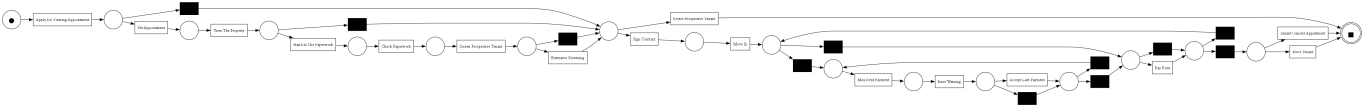
Page 14 / 20

Métrique	High	Medium	Low	Moyenne
Précision des Empreintes	1.00	1.00	1.00	1.00
Précision (Token-based Replay)	1.00	1.00	1.00	1.00
Généralisation	0.96	0.97	0.98	0.97
Simplicité du Réseau de Petri	0.69	0.70	0.70	0.70
Réseau de Petri Sonore	Non	Non	Non	Non

- **Fitness (0.99)** : Très proche du score parfait, il démontre sa capacité à s'adapter aux données observées.
- **Precision (TBR) (1)** : Il atteint une précision parfaite, ce qui signifie qu'il est extrêmement efficace pour éviter les comportements superflus ou imprévus. Cela en fait un excellent choix pour des processus nécessitant une modélisation stricte.
- **F-Score (0.98)** : Avec un F-Score presque parfait, il équilibre parfaitement précision et rappel, offrant une grande fiabilité pour des modèles de processus.
- **Simplicity (0.7)** : Il offre une simplicité relativement bonne, rendant les modèles compréhensibles pour les utilisateurs finaux.
- **Generalization (0.97) et Recall (0.97)** : Bien qu'il ne domine pas ces métriques, ses scores élevés montrent qu'il reste un choix polyvalent et compétent.

En somme, l'Heuristic Miner est un choix idéal pour des scénarios où la précision stricte et l'équilibre global (F-Score) sont essentiels, tout en restant performant sur les autres aspects. Bien qu'il soit plus adapté que l'Alpha Miner pour traiter des logs réels, il peut parfois produire des modèles plus complexes à interpréter et moins précis que des algorithmes plus avancés, comme l'Inductive Miner. Par exemple, il peut générer des "boîtes noires" ou des transitions silencieuses, compliquant ainsi l'analyse. Cependant, l'Heuristic Miner excelle dans des situations plus complexes, en prenant en compte des relations variées entre les événements, telles que la causalité et la concurrence, ce qui lui permet de mieux capturer des comportements complexes, y compris les répétitions d'événements.

Inductive



Sortie du réseau de Petri généré par Inductive Miner via PM4PY

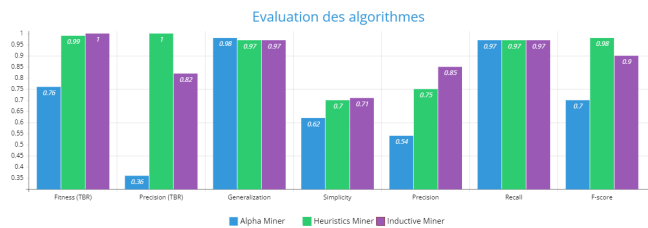
Métrique	High	Medium	Low	Moyenne
Fitness (TBR)	1.00	1.00	1.00	1.00
Precision (TBR)	0.82	0.81	0.82	0.82
Generalization	0.97	0.97	0.97	0.97
Simplicity	0.71	0.71	0.71	0.71
Soundness	True	True	True	True

Métrique	High	Medium	Low	Moyenne
Precision	0.85	0.84	0.85	0.85
Recall	0.97	0.97	0.97	0.97
F-score	0.90	0.90	0.90	0.90

- **Fitness (1)** : Avec un score parfait de 1, il parvient à reproduire complètement les traces observées, ce qui en fait une option fiable pour des processus où l'adéquation est essentielle.
- **Simplicity (0.71)** : Il obtient la meilleure simplicité parmi les trois, signifiant que le modèle généré est facilement compréhensible et utilisable, ce qui est souvent crucial pour l'analyse métier.
- **Precision (0.85)** : Avec une précision élevée, il montre sa capacité à éviter les comportements qui ne sont pas observés dans les données, réduisant ainsi le bruit.
- **Generalization (0.97)** : Bien qu'il ne surpasse pas Alpha Miner sur cette métrique, son score reste très élevé, indiquant qu'il trouve un bon équilibre entre spécificité et généralisation.
- **Recall (0.97)** : Son rappel élevé garantit qu'il couvre bien les traces observées.
- **F-Score (0.9)** : Ce score élevé reflète son excellent équilibre entre précision et rappel, confirmant sa robustesse globale.

En résumé, l'Inductive Miner est un excellent choix pour ceux qui recherchent un algorithme performant sur plusieurs dimensions clés, notamment la simplicité et la précision, tout en offrant un bon compromis pour des métriques comme la généralisation et le rappel. Cet algorithme, qui construit des modèles à partir de logs d'événements de manière inductive, identifie des structures de contrôle complexes telles que les séquences, les boucles, la concurrence et les choix. Il génère des modèles hiérarchiques et robustes, même en présence de bruit, garantissant des résultats corrects. L'Inductive Miner est particulièrement adapté pour modéliser des processus d'affaires complexes, avec des comportements imbriqués ou répétitifs, tout en restant rapide et efficace.

Comparaison des modèles



Concernant la soundness des modèles, voici les résultats pour les algorithmes suivants:

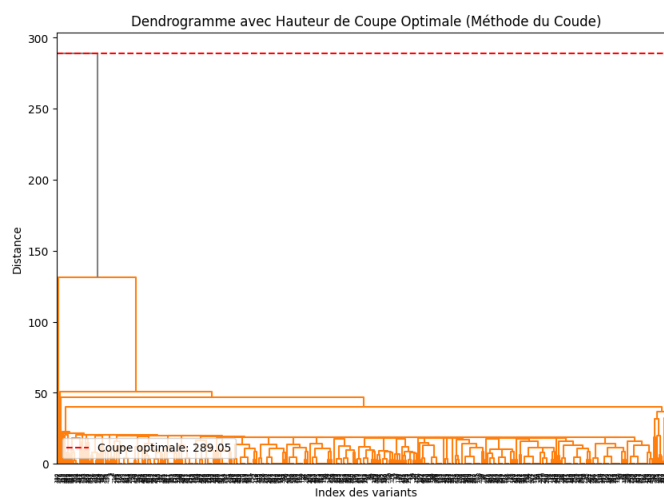
Algorithme	Soundness
Heuristics Miner	False
Inductives Miner	True

Le choix de l'algorithme optimal pour analyser le dataset Renting dépend fortement des priorités de l'étude. Inductive Miner se distingue par son excellent équilibre entre précision, simplicité et flexibilité, ce qui en fait une option idéale pour des analyses qui nécessitent des modèles interprétables et efficaces, notamment dans un contexte commercial. Il offre également de solides performances en termes de généralisation et de rappel,

garantissant une bonne couverture des comportements observés. Heuristics Miner, en revanche, excelle lorsqu'une modélisation précise des étapes du processus est cruciale. Il est particulièrement adapté pour des analyses nécessitant une stricte fidélité aux données d'origine, comme celles visant à optimiser ou valider des processus complexes. Enfin, Alpha Miner est plus pertinent pour des analyses macroscopiques où la détection des tendances générales ou des comportements utilisateur à long terme est privilégiée. Bien qu'il manque de précision et de simplicité comparé aux autres algorithmes, il reste une option viable pour des besoins de généralisation. En somme, le choix de l'algorithme dépend des objectifs spécifiques : Inductive Miner pour un modèle équilibré et interprétable, Heuristics Miner pour une modélisation détaillée et précise, et Alpha Miner pour une approche axée sur la généralisation.

Clustering et Profils des Groupes

Afin de déterminer distinctement les facteurs discriminants et effectuer des prédictions pour déterminer si un profil d'une nouvelle personne est susceptible de signer un contrat d'appartement, nous avons effectué un clustering sur nos données.



Dendrogramme pour sélectionner la coupe optimale (optimal = 2)

Note : À partir d'ici, nous n'avons strictement rien compris à nos manipulations. Malgré l'application des connaissances acquises en cours, toute l'aide extérieure, aussi bien au près de camarades que des IA, nous n'avons pas réussi à obtenir les résultats que nous espérions... Les résultats sont à peine explicables et nous aurions aimé aller plus loin avec ces derniers.

Maintenant que nous connaissons la coupe optimale, il est temps de découper nos données. Pour déterminer des clusters sur nos jeux de données, nous avons effectué plusieurs approches :

- **AgglomerativeClustering**
 - Cette approche n'a pas pu se poursuivre pour des raisons techniques **MemoryError: Unable to allocate 3.77 GiB for an array with shape (4047435406,) and data type bool**
- **Kmeans**
 - Nous avons effectué plusieurs tests via plusieurs approches dans nos codes, mais rien n'a donné de modèles satisfaisants. Prenons par exemple 2 graphes générés par nos codes **test_kmeans.py** et **test_kmeans_2.py**...

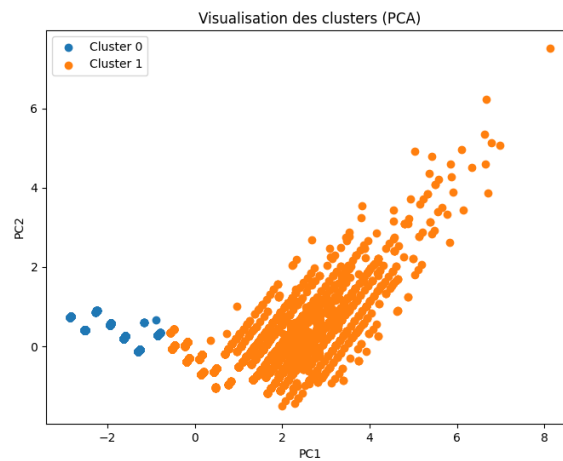


Figure des clusters générés avec `test_kmeans.py`

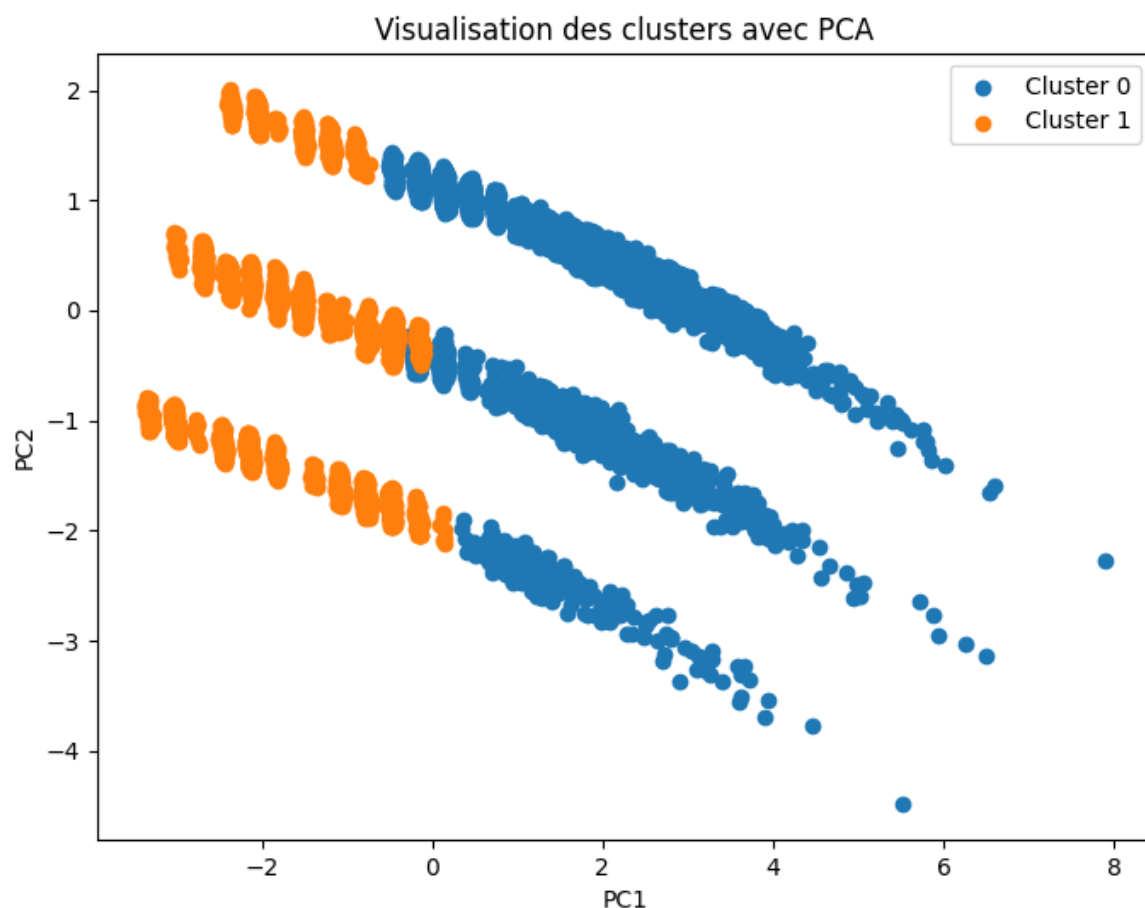
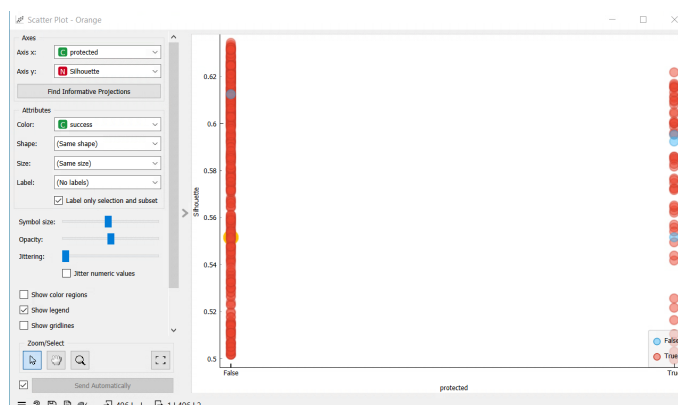


Figure des clusters générés avec `test_kmeans_2.py`

Dans les tests, on effectue une prédiction sur une nouvelle trace pour savoir si elle a ses chances d'avoir un appartement ou non, basé sur l'évènement "Sign Contract". Et dû au cluster à la proportion fort discutable, la prédiction n'a pas trop de sens puisque tous les points finissent dans le même cluster.

Nous sommes également passé par une approche à l'aide de variants pour réduire le nombre de données à traiter, mais même là ce n'est pas pertinent, car les refus de contrats sont des actions souvent répétées en quelques étapes, donc au total on peut avoir ~4 variants sur +400 cas où une personne loge dans un

appartement. Nous avons aussi utilisé l'outil "Orange" en Python pour analyser nos données, et c'est là qu'on s'est rendu compte que cette approche était un peu



Affichage généré par Orange de répartition des variants selon la variable "protected", colorié selon "Sign Contract"

Quelque chose qu'on aurait pu faire pour améliorer nos analyses, c'est **diviser le processus métier en 2 sous parties** : L'un pour **l'acceptation de contrat**, l'autre **pour les résidents**. Mais ça n'a pas pu se faire à défaut de temps.

Gestion de projet

Lors de ce projet, nous avons choisi de créer un dépôt GitHub pour centraliser notre code et gérer nos tâches. Pour ce faire, nous avons utilisé les issues et les milestones afin de classer et suivre l'avancement des différentes tâches de manière organisée.

De plus, nous avons eu recours à Discord pour faciliter la communication entre les membres de l'équipe, permettant ainsi un échange rapide et fluide d'informations.

Réflexion sur la Vie Privée

Le jeu de données repose sur des informations réelles qui ont été anonymisées. Cet anonymat empêche toute identification des individus concernés, garantissant le respect de leur vie privée, grâce à l'utilisation d'identifiants (*id*) plutôt que de stocker en clair le nom des personnes suivies. Si les données n'avaient pas été anonymisées, elles auraient pu inclure des informations sensibles, comme le nom ou l'apparence, ce qui aurait impliqué une responsabilité accrue dans leur gestion et leur utilisation.

Conclusion

Nos observations montrent qu'il n'y a pas de réelles discriminations entre les données et que le résultat de "Sign Contract" n'est pas plus influencé par l'âge, le temps d'étude, ou si la personne parle allemand par exemple.. D'autant qu'il manque en plus la variable d'appartenance d'ethnie, qui aurait pu donner plus d'indications s'il y a une réelle discrimination.

Comme dit dans la partie cluster, nous aurions pu diviser le processus métier en 2 sous parties : L'un pour l'acceptation de contrat, l'autre pour les résidents. Puisque ça n'a pas trop de sens d'étudier le suivi d'un

processus et de le raccourcis vers une fin distincte sur le temps comme là.

Références

- [\(Un\)Fair Event Logs](#)
- [PM4PY](#)
- [ProM](#)